

Q7(a). $w \text{ update} = w - \text{eta} * (p - y) x$; $b \text{ update} = b - \text{eta} * (p - y)$. The gradient is averaged over the batch.

Q7(b). lower threshold catches more fraud, so recall rises. Precision usually falls and false positives rise because more normal cases are flagged.

Q2(a). Let $a = \log n$. The recursion tree has a level cost $n(a - i)$, so the sum is $n[a + (a-1) + \dots + 1]$.

Q13. bonus: the algorithm should be tested on a disconnected graph too.

Q2(a). continued: this is $n \log^2 n$, so $T(n) = \Theta(n \log^2 n)$, assuming the leaf work is linear or smaller.

Q4(b). serializable behaves like one-at-a-time execution. Snapshot isolation reads a stable version but two transactions can both read a free seat and then write different rows: write skew.

Q3. distances from s are $(0, 3, 2, 4, 7)$. predecessors: $a = b$, $b = s$, $c = a$, $d = c$.

Q1. Potential can be stored as twice the number of empty slots. A push that does not resize has amortized cost one; the expensive resize is paid by the potential saved by earlier pushes. Shrinking at one quarter avoids oscillation.

Q6(a). Assume a pumping length p and choose $0^p 1^p$. Any split xyz with $|xy| \leq p$ has y only zeros. Pumping y changes the zero count but not the one count, so the result is not in L .

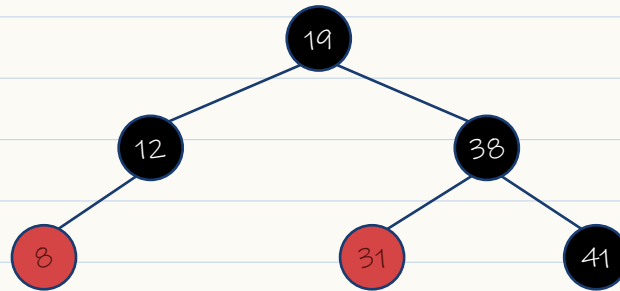
Q6(b)(i). push each 0 onto the stack; for each 1 pop one 0; accept when the input ends and the stack has only the bottom marker.

Q11(a). POST /payments takes amount, currency and an Idempotency-Key. Store the first result by key; a replay returns the same status and body without charging again.

Q4(a). CAP says a partition forces a choice between consistency and availability. A quorum can favour consistency; accepting writes on either side favours availability.

Q8.

after rotations: black root 19



Q12. $\text{accuracy} = (168 + 1758) / 2000 = 0.963$
 $\text{precision} = 168 / (168 + 42) = 0.800$
 $\text{recall} = 168 / (168 + 32) = 0.840$
 $F1 = 2 * 0.800 * 0.840 / (0.800 + 0.840) = 0.819$

Q5. WITH totals AS (SELECT customer_id, region, SUM(amount) total FROM Orders JOIN Customers USING(customer_id) WHERE created_at >= CURRENT_DATE - 30 GROUP BY customer_id, region) SELECT * FROM (SELECT totals.*, DENSE_RANK() OVER (PARTITION BY region ORDER BY total DESC) r FROM totals) z WHERE r <= 2;

Q10. problem one: admin is not enough validation; allow-list the path and reject traversal. problem two: request.body is written without a size/type check; validate and limit it before storage.

Q9. skipped - no answer

(intentional unanswered case)